STRACTURED PROGRAMMING
CS 6135

Introduction to programming

A computer is a programmable machine. This means it can execute a programmed list of instructions and respond to new instructions that it is given.

A computer Program is a step by step set of instructions that a computer has to work through in a logical sequence in order to carry out a particular task.

Computer Programming is the process of developing and implementing various sets of instructions to enable a computer to do a certain task.

Programmers translate the solutions or tasks into a language the computer can understand.

Programming Languages

A Programming language is the vocabulary and set of grammatical rules used by people to write instructions for the computer to perform specific tasks.

- There are many different programming languages each having a unique set of keywords(words that it understands) and a special syntax (grammar)for organizing program instructions.

Examples of programming languages are: *Java, C#,PHP, Python,C, C++, Visual basic, Javasript, Ruby*

Levels of programming languages

Programming languages are classified into two levels:

1. Low Level Programming language
2. High Level Programming language

1. Low Level Programming language

The low level language is machine language or very close to machine language.

Low level languages are categorized into two categories:-

i. Machine languages
ii. Assembly languages

Machine language

The machine language is low level language that writes programs using the machine code of 1s and 0s, which is directly understood by the computer.

Machine Language is basically the only language which computer can understand. In fact manufacturer design computer to obey just one Language (machine language 1 and 0).

1		00000000	00000100	0000000000000000
2	01011110	00001100	11000010	0000000000000010
3		11101111	00010110	0000000000000101
4		11101111	10011110	0000000000001011
5	11111000	10101101	11011111	0000000000010010
6		01100010	11011111	0000000000010101
7	11101111	00000010	11111011	0000000000010111
8	11110100	10101101	11011111	0000000000011110
9	00000011	10100010	11011111	0000000000100001
10	11101111	00000010	11111011	0000000000100100
11	01111110	11110100	10101101	
12	11111000	10101110	11000101	0000000000101011
13	00000110	10100010	11111011	0000000000110001
14	11101111	00000010	11111011	0000000000110100
15		01010000	11010100	0000000000111011
16			00000100	0000000000111101

Assembly language

Assembly language is low level symbolic language written using **mnemonics** (abbreviated sets of letters) or short codes that suggest their meaning and are therefore easier to remember. The instructions of the Assembly language will also be converted to machine codes by language translator called **assembler** to be executed by the computer.

An example of a program code to add and store two numbers would be:

LDAA, 20 *:load accumulator A with the value 20*
ADD A,10 *: add the value 10 to accumulator A*
STOB, A *: store contents of accumulator A into storage register B*
NOP *:no operation (stop here)*

Advantages of low level languages

- Very fast to execute because it is already in the language that the computer can understand.
- Require little memory resources takes up less storage space.
- No need of language translator for machine language.
- Useful for writing system programs where accuracy is required

Disadvantages of low level languages

- Difficult to interpret by the programmer (requires the aid of a reference manual to interpret the meaning of each code)
- Easy to make mistakes in the sequence of 1s and 0s; replacing a 1 for a 0 can result in the wrong command/instruction being executed
- It is difficult to identify mistakes made
- Time-consuming, slow and tedious to write
- Machine dependent, e.g. one written for an IBM machine cannot work on an Apple machine.

High-level programming languages

High level programming language is a language that is near to natural language, therefore it is machine independent and uses variables and objects, Boolean expressions, functions, loops, threads.

High-level languages permit faster development of large programs. The final program as executed by the computer is not as efficient, but the savings in programmer time generally far outweigh the inefficiencies of the finished product.

Programs written in a high-level language must be translated into machine language by a compiler or interpreter.

Advantages of high level programming languages

- High-level language programs are easy to debug
- They are machine independent. Provide programs that can be used on more than one computer.
- They are user friendly and easy to learn because they are near to natural language.
- They are flexible hence they enhance the creativity of the programmer, increasing productivity
- Allows the programmer to focus on understanding the user's needs and design the required software.
- They permit faster development of large programs.

Disadvantages of high level programming languages

- They are executed much slower than low-level programming languages
- They have to be translated into machine code before execution
- Require more memory than the low level languages

Key terminologies..

Program Syntax is the spelling and grammar of a programming language. (Each program defines its own syntactical rules that control which words the computer understands, which combinations of words are meaningful, and what punctuation is necessary.

Computer language semantics is the meaning of the language's features. Semantics describes the processes a computer follows when executing a program in that specific language.

Key terminologies..

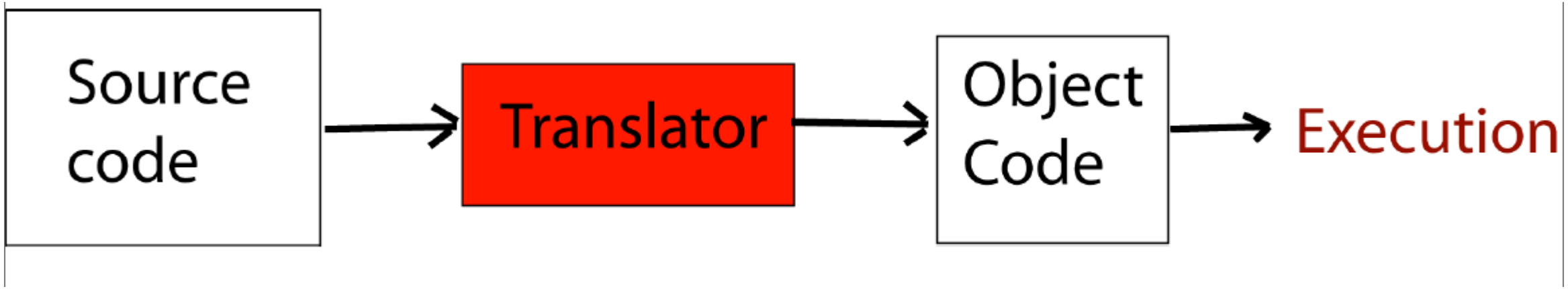
Source code is a Program instruction written as text file by the programmer, that must be translated by a compiler or interpreter or assembler into an object code before execution. Source code cannot be understood by the computer until it has been translated into machine code.

Object code is a program code in machine language that is ready for execution by the computer.

Language Translators

Language translators are system programs that convert assembly language and high-level language to machine language for the program to execute because Computers work in machine code only.

Programs must be translated into machine/Object codes before execution



Types of translators

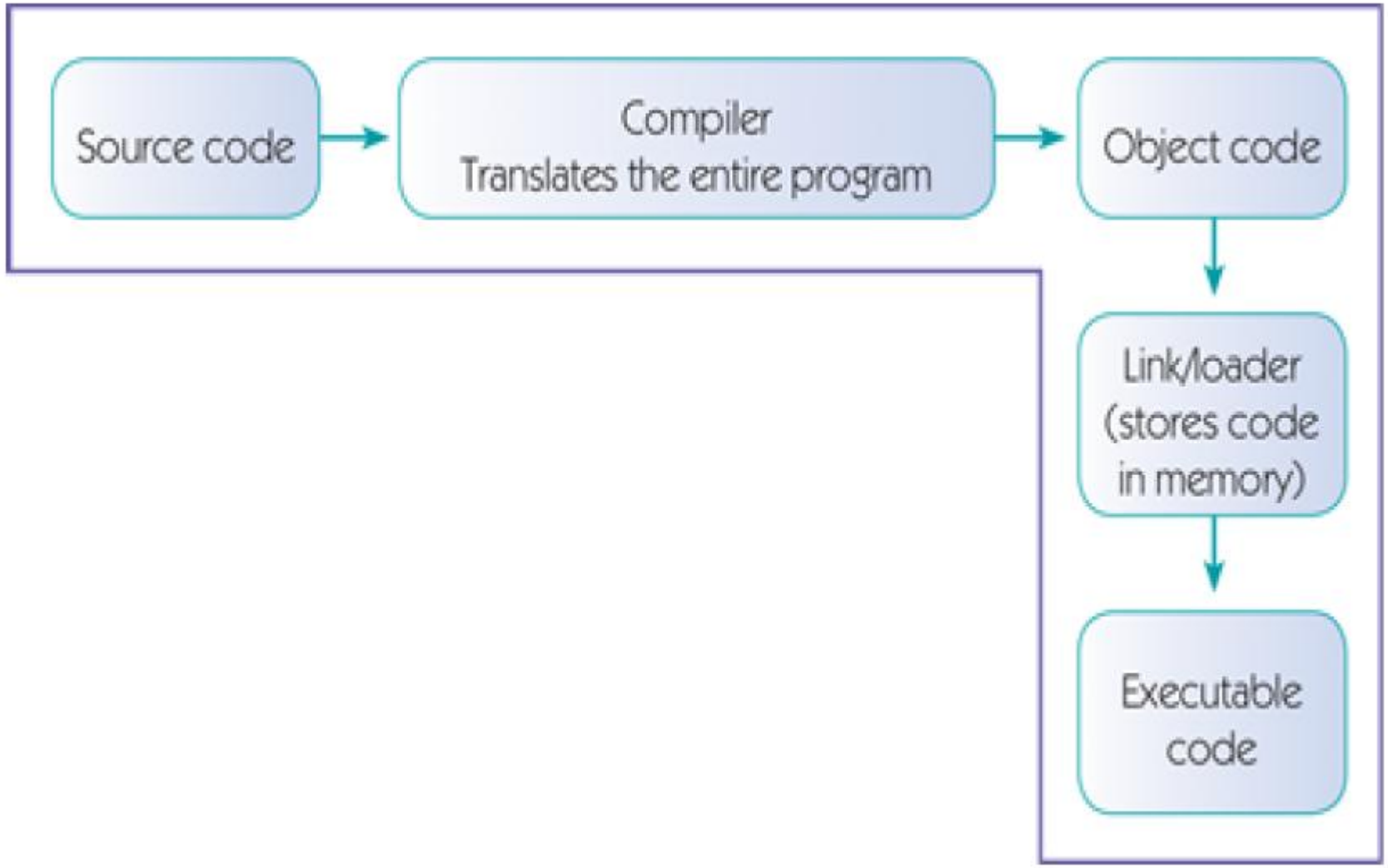
There are three types of translators:

1. Assemblers
2. Compilers
3. Interpreters

Assembler is a language that translates an assembly-language program into machine code.

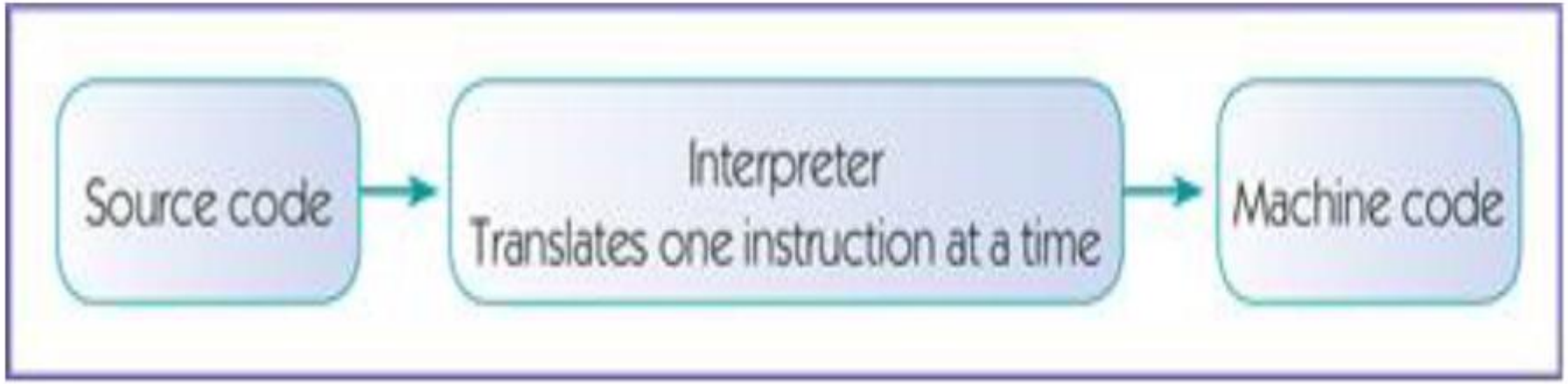
A compiler is a computer program that transforms the entire source code written in a high level programming language into object code for execution.

How a Compiler works



How an Interpreter works

An interpreter is a computer program that directly executes a high level programming language one line at a time to run each time the source code is run because no object code is generated.



Integrated Development Environment (IDE)

Integrated Development Environment is a software program that is designed to help programmers and developers build software.

Most IDEs include:

- a source code editor
- a compiler
- build automation tools
- a debugger

Debugger *A special program used to find errors (bugs) in other programs. A debugger allows a programmer to stop a program at any point and examine and change the values of variables.*

Algorithms, Pseudocode and Flowcharts

Performing a Task on The Computer

- The first step in writing instructions to carry out a task is to determine what the output should be that is, exactly what the task should produce.
- The second step is to identify the data, or input, necessary to obtain the output.
- The last step is to determine how to process the input to obtain the desired output, that is, to determine what formulas or ways of doing things can be used to obtain the output.

Program Development Cycle

1. Analyze: Define the problem.

Be sure you understand what the program should do, that is, what the output should be. Have a clear idea of what data (or input) are given and the relationship between the input and the desired output.

2.Design: Plan the solution to the problem.

Find a logical sequence of precise steps that solve the problem. Such a sequence of steps is called an algorithm. Every detail, including obvious steps, should appear in the algorithm.

Program Development Cycle

3. Choose the interface: Select the objects.
Determine how the input will be obtained and how the output will be displayed. Then create appropriate commands to allow the user to control the program.

4. Code: Translate the algorithm into a programming language.
Coding is the technical word for writing the program.

5. Test and debug: Locate and remove any errors in the program.
Testing is the process of finding errors in a program, and debugging is the process of correcting errors that are found. (An error in a program is called a bug.)

Program Development Cycle

6. Complete the documentation: Organize all the material that describes the program.

Documentation is intended to allow another person, or the programmer at a later date, to understand the program. Internal documentation consists of statements in the program that are not executed, but point out the purposes of various parts of the program.

Documentation might also consist of a detailed description of what the program does and how to use the program (for instance, what type of input is expected).
For commercial programs, documentation includes an instruction manual. Other types of documentation are the flowchart and pseudocode

Key terminologies..

An algorithm is a list of instructions, procedures, or formulas used to solve a problem.

Step 1: Start
Step 2: Declare variables num1, num2 and sum.
Step 3: Read values num1 and num2.
Step 4: Add num1 and num2 and assign the result to sum.
sum←num1+num2
Step 5: Display sum
Step 6: Sto

Pseudocode is a narrative/descriptive form using human language statements to describe the logic and processing flow of a program.

A flowchart is a diagrammatic/graphical representation of a program's algorithm using standard symbols and short statements to describe various activities.

Pseudo-code commonly used keywords

Pseudo-code uses several keywords to indicate common input, output, and processing operations:

- **Input: READ, READLINE, OBTAIN, GET, ENTER, INPUT**
- **Processing: Initialize, SET, INIT Add one, INCREMENT, BUMP, COMPUTE, CALCULATE, DETERMINE**
- **Output: PRINT, DISPLAY, SHOW, WRITE, WRITELINE**

```
Start
Read A, B
Calculate C = A*B
Display C
Stop
```

Example Pseudocode

Program flowchart symbols

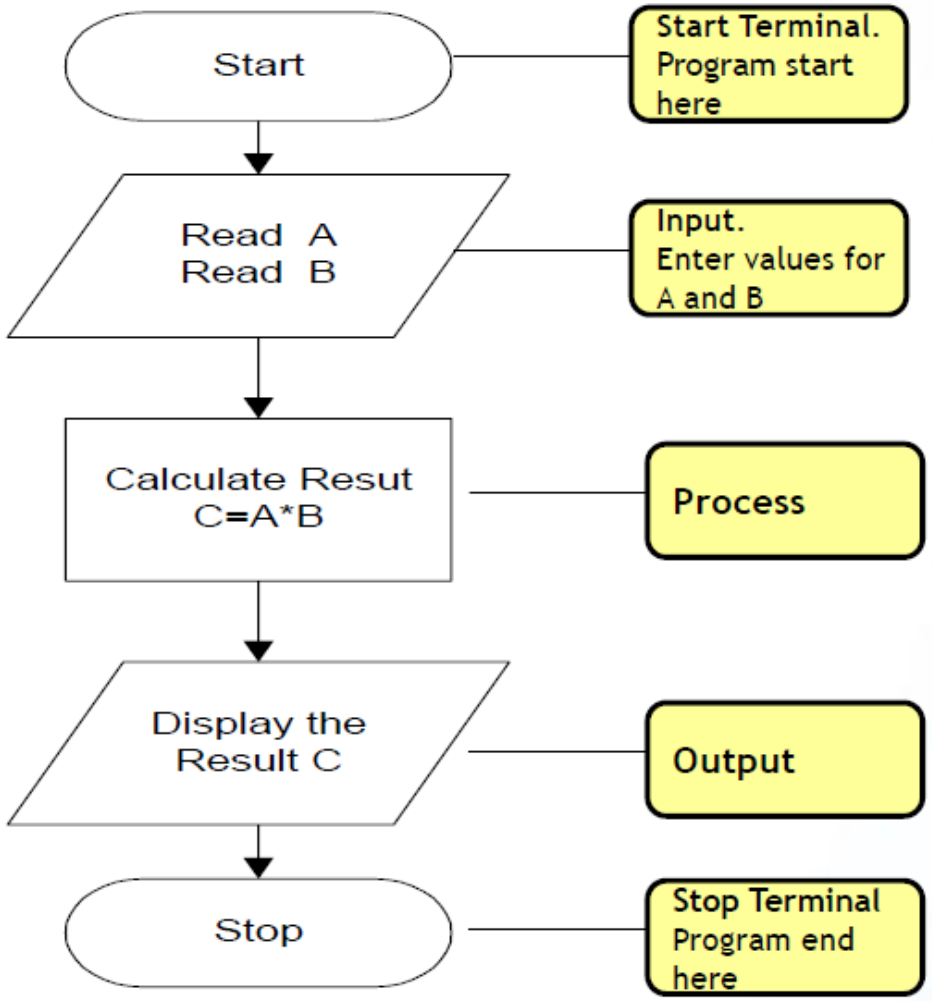
These symbols are called ANSI symbols (called after the American National Standards Institute that developed them).

A flowchart shows how a program works before one begins the actual coding of the program.

flowchart symbols

Symbol,	Name,	Meaning
	Flowline	Used to connect symbols and indicate the flow of logic.
	Terminal	Used to represent the beginning (Start) or the end (End) of a task.
	Input/Output	Used for input and output operations, such as reading and printing. The data to be read or printed are described inside.
	Processing	Used for arithmetic and data-manipulation operations. The instructions are listed inside the symbol.
	Decision	Used for any logic or comparison operations. Unlike the input/output and processing symbols, which have one entry and one exit flowline, the decision symbol has one entry and two exit paths. The path chosen depends on whether the answer to a question is “yes” or “no.”

Example Flowchart



Structured Programming

Structured programming was first suggested by Corrado Bohm and Guiseppe Jacopini. The two mathematicians demonstrated that any computer program can be written with just three structures: **sequences, decisions, and loops.**

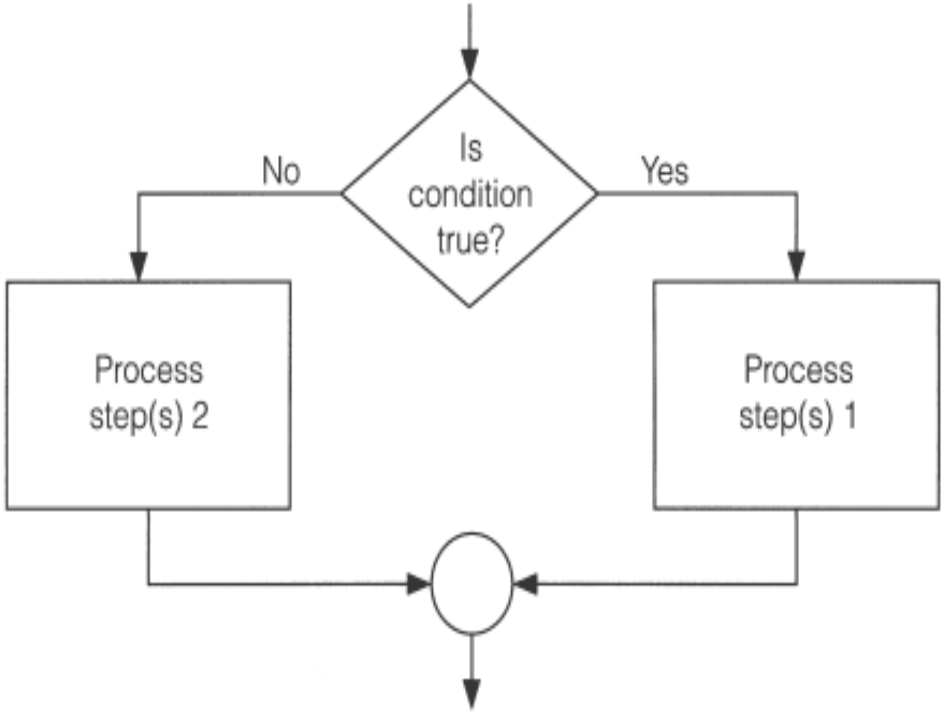
- **Sequences:** one command is executed after previous one.
- **Decisions (selections):** statement(s) is (are) executed if certain condition gives TREU or FALSE value.
- **Loops (repetition):** statement(s) is (are) executed repeatedly until certain condition gives TREU or FALSE value.

Pseudocode and Flowchart for a Decision Structure

Three selection structure in C++ programming:

- If
- If –else
- Switch

```
If condition is true Then
    Process step(s) 1
  Else
    Process step(s) 2
End If
```



Example

Write pseudocode and algorithm to determine a student's average grade and indicate whether he is successful or not and draw a flowchart.

- The average grade is calculated as the average of mid-term and final marks.
- Student will be successful if his average grade is grater or equals to 60.

Solution

Pseudocode

Start
Use variables mid term , final and average
Input mid term and final
Calculate the average by summing mid term and final and dividing by 2
if average is below 60
Print “FAIL”
else
Print “SUCCESS”
Stop

Solution

Detailed Algorithm

1. Step: Input mid-term and final
2. Step: average =(mid-term + final)/2
3. Step: if (average < 60) then

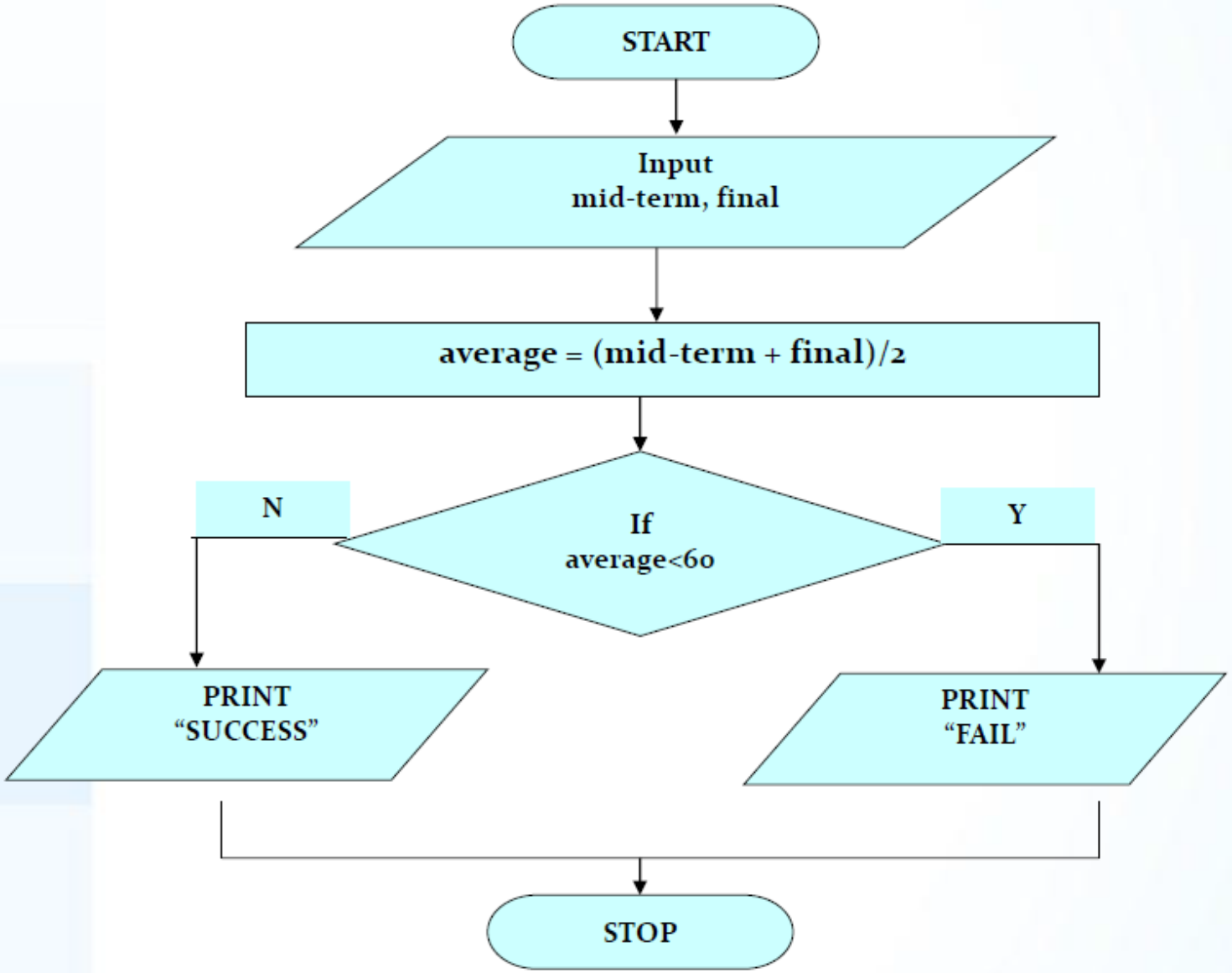
    Print “FAIL”

    else

    Print “SUCCESS”

    endif

Flowchart

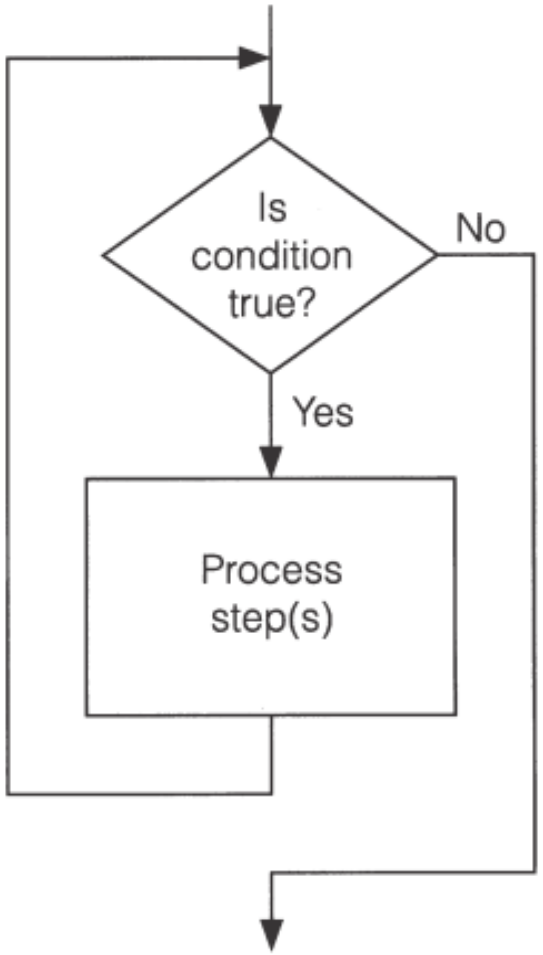

START
Input
mid-term, final
average = (mid-term + final)/2
If
average<60
N
Y
PRINT
"SUCCESS"
PRINT
"FAIL"
STOP

Pseudocode and Flowchart for a Loop

Three repetition structure in C++ programming:

- While
- Do –while
- For

```
Do While condition is true
    Process step(s)
Loop
```



Example

Write an algorithm which calculates the average exam grade for a class of 5 students.

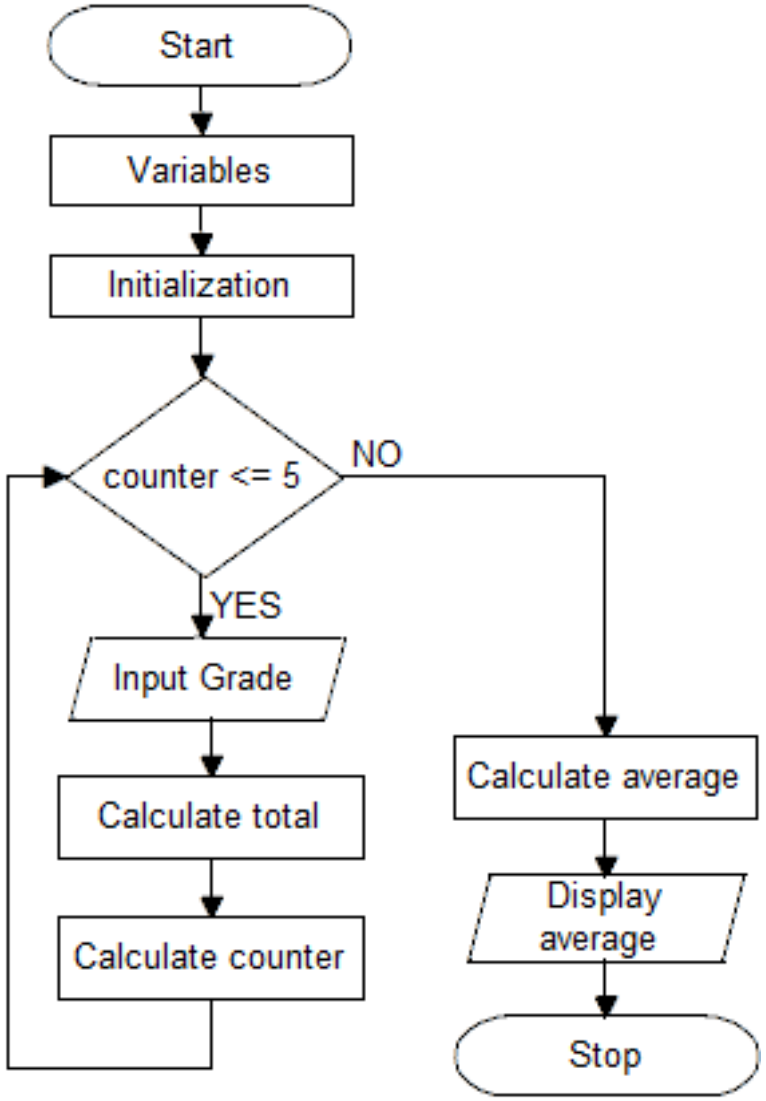
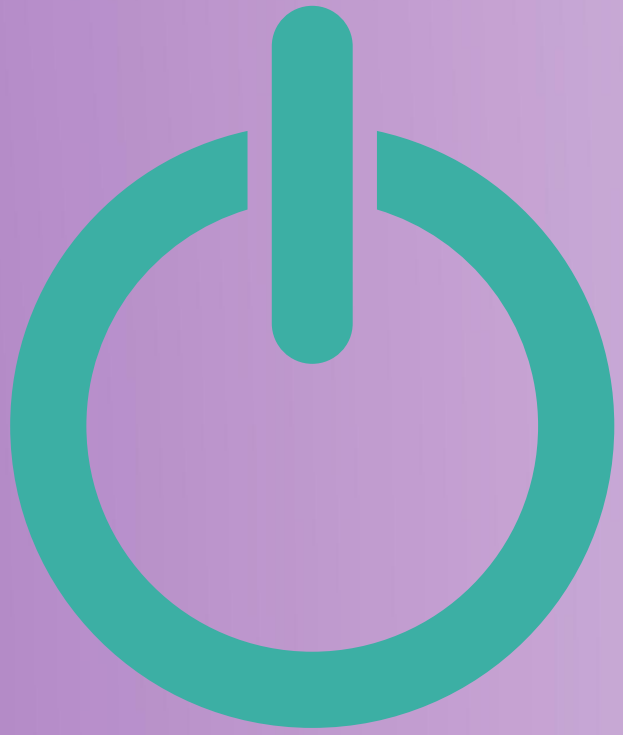
- What are the program inputs? the exam grades
- Processing: Find the sum of the grades; count the number of students; (counter controlled), calculate average grade = sum of grades / number of students.
- What is the program output? the average exam grade

Solution

Pseudocode

```
Start
Use variables total, counter, grade, average
Initialize total= 0
Initialize counter= 1
While (counter <= 5)
Input grade
Calculate total = total + grade
Calculate counter = counter+ 1
End-while
Calculate average= total / 5
Display average
Stop
```

flowchart


Start
Variables
Initialization
counter <= 5
NO
YES
Input Grade
Calculate total
Calculate counter
Calculate average
Display average
Stop